

Phase 1 — Intake & Valuation · Engineering Plan

Audience: an implementing agent. This is an execution spec, not a discussion doc. Every open architectural question has been resolved into a **Locked Decision** below — do not re-litigate them; implement them. When a decision conflicts with reality you discover in the code, stop and flag it, don't silently diverge.

Source product docs: plans/Scorta Brokerage Build.md, plans/Phase 1 — Intake & Valuation.md (this plan operationalizes them). Phases 2–3 docs exist but are **out of scope** here.

Engineering contract: .claude/CLAUDE.md (green build, Zod-validated input, unhappy-path handling, RLS-in-migration, tests for lib/ logic, no process.env outside env.mjs, server-only db, @/* alias). Everything below must satisfy it. This is a **real product**, not a demo.

Timeline: 2–3 days. Scope is sequenced as a working spine first, depth second, with explicit cut-lines. **Ceiling: Phase 1 only — intake → signed valuation opinion letter.** No CIM.

0. The one-paragraph summary

A seller completes the public **ExitIQ assessment** (already live) and gets a GAP report. After an off-platform consultation call, a Scorta operator **creates the seller's account and a deal record** linked to that assessment. The seller logs into the authenticated workspace, **signs the engagement + confidentiality agreements** (real click-sign), and **uploads their financial and operational documents**. A pipeline of agents **classifies** each document, **extracts** structured financials, scores **completeness** against a broker requirement catalog (deterministic), and **normalizes** everything into a single versioned **financial model** with proposed **add-backs**. The seller **approves the recast** (finalizing SDE), then a Valuation agent generates a formal **Valuation Opinion Letter** (precise SDE × industry multiple, methodology disclosed). The seller **approves the valuation range**, which moves the deal to phase2_ready. That is the entire Phase 1 deliverable.

1. Current-state ground truth (read before building)

1.1 What is real and is the foundation we build on

- **Public assessment** (components/exitiq/*, app/api/assessment/*): stage 1 + email gate live; stages 2–4 schema-ready. Writes `assessment_sessions`; generates `assessment_reports` (GAP report) via `streamText` + Sonnet in `app/api/assessment/generate/route.ts`. **Keep**.
- **Auth** is real Supabase Auth (`signInWithPassword` in `components/scorta/LoginPanel.tsx`). `app/(app)/layout.tsx` guards via `supabase.auth.getUser()` → `redirect("/login")`. **No public signup** — users are operator-provisioned. **Keep; build on it**.
- **DB access pattern**: `lib/db/index.ts` exports a Drizzle singleton over the `postgres.js` transaction-pooler connection (`prepare: false`). API routes/server code read/write via Drizzle, which **bypasses RLS**. RLS exists only to gate the anon role for inserts.
- **Persist-after-respond pattern**: `session/route.ts` and `generate/route.ts` return `200/stream` first and do DB writes inside `after()`. Mirror this for slow agent work.
- **Existing real valuation primitives (reuse, don't reinvent)**: `lib/assessment/industries.ts` → `INDUSTRIES[].sdeMultiple: [low, high]` + `findIndustry()`; `lib/assessment/scoring.ts` → `getValuationRange, isSBAEligible, fmt`; `lib/assessment/sba.ts` → `computeSBASnapshot` (loan/down-payment/monthly/DSCR inputs). These are **bracket-based** (driven by self-reported revenue/SDE buckets). The brokerage valuation must feed the **precise normalized SDE** from ingestion into the same industry multiple band.

1.2 What is mock and must be IGNORED (do not extend, do not delete in this sprint)

- `lib/persona.ts` (PERSONA, STATIONS) — single hard-coded persona (Chandan Patel / Palace Kitchen & Catering). The `(app)` station components (`ConnectStation`, `IngestionStation`, `RecastStation`, `RiskStation`, `BoardroomStation`, `DocumentsStation`, `VDRStation`, `LendersStation`, `BuyersStation`, `OutreachStation`, `UploadStation`, `CASEChat`, etc.) render against PERSONA. They are **debt scaffolding**, explicitly out of scope. We do **not** retrofit them. We build clean, real Phase-1 surfaces and seed the same persona as **real DB rows** so the product is demoable with the familiar numbers.
- The `.exitiq-debug/` persona files and the old demo-sprint workflow are historical. Ignore.

1.3 What does not exist yet (we create it)

- No deals, sellers, documents, `financial_models`, agreements, approvals tables.
- No Supabase Storage usage anywhere (@supabase/supabase-js is installed; Storage is unused).
- No `seller_intent` captured anywhere.
- No `generateObject` usage; only `streamText` for the report.
- No PDF library, no file-upload library (we use native input + Storage signed URLs).

- No background queue. Orchestration will be synchronous routes + `after()` + status columns.

2. Locked decisions (every open question, answered)

#	Question	Locked Decision
D1	Where is the deal record created?	Operator action after the consultation call. Not a DB trigger on <code>assessment_sessions.completed_at</code> . Not every assessment becomes a deal. Mechanism: an operator-only server action <code>createDealFromAssessment(sessionId, sellerEmail)</code> .
D2	Schema linkage assessment ↔ deal	FK + selective denormalization. <code>deals.assessment_session_id</code> → <code>assessment_sessions.session_id</code> (nullable FK). Copy <code>seller_intent</code> , <code>business_name</code> , <code>industry</code> onto the deal at creation (deal becomes source of truth once real numbers land).
D3	Where does <code>seller_intent</code> live?	Both. First-class picker added to the assessment (new typed column <code>assessment_sessions.seller_intent</code> + a step in the ExitIQ flow), denormalized to <code>deals.seller_intent</code> at creation. Enum: <code>sell</code> · <code>understand_value</code> · <code>improve_valuation</code> · <code>get_financing</code> .
D4	Account model	Supabase Auth user = the seller. A sellers row maps <code>auth.users.id</code> → seller profile and is the owner of the deal. Operator-provisioned (no public signup) via an isolated admin module using the service-role key.
D5	Engagement letter / confidentiality e-sign	Generated documents + real in-app click-sign , not DocuSign. Store an immutable rendered snapshot in Storage; record <code>signed_at</code> , <code>signed_name</code> (typed), <code>signer_user_id</code> , <code>document_sha256</code> , ip in an <code>agreements</code> table. DocuSign is a future swap behind the same interface.
D6	Deal-structure preferences storage	Hybrid. Typed columns for the 5 decision-driving prefs (<code>asset_vs_stock</code> , <code>seller_note_tolerance</code> , <code>earnout_tolerance</code> , <code>real_estate</code> , <code>target_close</code>); JSONB <code>preferences_notes</code> for free text.
D7	Document upload surface	New real surface. New route <code>app/(app)/intake/</code> backed by Supabase Storage + a <code>documents</code> table. The mock <code>/upload</code> , <code>/documents</code> , <code>/vdr</code> are ignored.
D8		

#	Question	Locked Decision
	Storage strategy & ownership	One private Supabase Storage bucket deal-documents. Object path deals/{dealId}/{documentId}/{filename}. Each upload ⇒ a documents row owned by deal_id . Server mints signed upload/download URLs after an ownership check.
D9	Completeness output schema	Requirement catalog in code (lib/brokerage/requirements.ts) with severity: deal_blocking · important · nice_to_have. Per-deal status is computed deterministically from the documents rows; the resulting gap list is snapshotted to deals.completeness (JSONB) and rendered with severity flags + auto-drafted seller requests. No LLM in the scoring math.
D10	Normalization output schema	Versioned financial_models table holding a typed FinancialModel JSON (per-year P&L lines, reported EBITDA, addBacks [], normalized SDE/year, TTM, basis, flags). This is the canonical structured shape downstream (recast/CIM) consume.
D11	Valuation opinion: section or doc?	Separate formal document (its own deliverables/documents row + approval gate), generated by a dedicated Valuation agent from the precise financial model. The assessment GAP report (assessment_reports) stays as the intake artifact.
D12	Valuation seller-approval flow	Hard gate, not informational. Seller approves the range on the opinion letter; approval records agreed SDE + range + asking-price anchor on the deal and flips status → phase2_ready (matches Phase 2's hard gate "valuation range agreed by seller").
D13	Which steps are LLM vs rule-based?	Classification = Haiku generateObject . Extraction = Sonnet generateObject (accuracy-critical). Completeness = deterministic code (LLM only drafts request copy, Haiku). Normalization = Sonnet generateObject proposes structure + add-back candidates, but all arithmetic is recomputed in code (never trust LLM math). Valuation letter prose = Sonnet streamText with numbers injected from code .
D14	Approval-gate shape per step	Three hard gates: (1) agreements signed; (2) recast/add-back schedule approved (finalizes SDE); (3) valuation range approved. Classification/extraction/normalization auto-run and surface an editable review , not a hard block. Completeness is a deterministic gate: deal cannot advance to recast until all deal_blocking requirements are satisfied.

#	Question	Locked Decision
D15	Agent event triggers	Deal status state machine + synchronous orchestration. <code>document.uploaded</code> → <code>classify</code> (<code>after()</code>); <code>classification done</code> → <code>extract</code> ; <code>documents change</code> → <code>recompute completeness</code> ; <code>deal_blocking cleared</code> → <code>enable normalization</code> ; <code>recast approved</code> → <code>enable valuation</code> ; <code>valuation approved</code> → <code>phase2_ready</code> . No queue; durability lives in <code>deals.status</code> + <code>documents.status</code> (+ idempotent re-runs).
D16	Consultation call modeled in-app?	Off-platform. Modeled only as deal status (<code>consultation_scheduled</code> → <code>onboarding</code>) + a scheduling CTA. The operator sets the state. Not an in-app calendar flow.
D17	Minimum to unblock Phase 2	Seller intent + deal record + a populated FinancialModel with approved SDE + an approved valuation range. The CIM agent (Phase 2) consumes <code>financial_models</code> + <code>deals</code> only.
D18	Tenancy	Real multi-seller schema, seeded with the persona. Chandan/Palace Kitchen becomes deal #1 in the DB. The (app) layout loads the real deal for the authed user and feeds AppShell real identity (begins unwinding the PERSONA coupling — contained to identity/header fields).

3. Target data model

3.1 New schema file — `lib/db/schema/brokerage.ts`

Export all tables from `lib/db/schema/index.ts`. Types live in `lib/brokerage/types.ts` and are referenced via `.$type<...>()` on JSONB columns (mirror the `assessments.ts` convention).

```
// lib/brokerage/types.ts (pure types – no imports from app/ or db)

export type SellerIntent = "sell" | "understand_value" |
  "improve_valuation" | "get_financing"

export type DealStatus =
  | "consultation_scheduled" // created from assessment, pre-
    onboarding
  | "onboarding" // account created; agreements
    pending
  | "ingestion" // agreements signed; collecting
    documents
  | "recast_pending" // deal-blocking docs satisfied;
    model drafted, awaiting seller sign-off
  | "valuation_pending" // recast approved; opinion letter
    generating/awaiting approval
```

```

|
    "phase2_ready" // valuation range approved –
    Phase 1 complete

export type AssetVsStock = "asset" | "stock" | "undecided"
export type Tolerance = "yes" | "no" | "maybe"
export type RealEstate = "owns" | "leases" | "none"

export type DocumentCategory =
  | "tax_return" | "pnl" | "balance_sheet" | "bank_statement" |
    "ar_aging" | "ap_aging"
  | "payroll" | "lease" | "equipment_list" | "inventory" |
    "customer_contract"
  | "vendor_contract" | "org_chart" | "entity_docs" | "insurance"
  | "license_permit"
  | "other" | "unclassified"

export type DocumentStatus =
  | "uploaded" | "classifying" | "classified" | "extracting" |
    "extracted" | "failed"

export type RequirementSeverity = "deal_blocking" | "important" |
  "nice_to_have"

export interface CompletenessGapItem {
  requirementKey: string
  label: string
  severity: RequirementSeverity
  satisfied: boolean
  satisfiedByDocumentIds: string[]
  requestCopy?: string // Haiku-drafted seller-facing ask
}

export interface CompletenessSnapshot {
  computedAt: string
  dealBlockingSatisfied: boolean
  score: number // 0-100, weighted by severity
  gaps: CompletenessGapItem[]
}

export type AddBackStatus = "proposed" | "approved" | "rejected"
export type AddBackCategory =
  | "owner_compensation" | "owner_perk" | "one_time" |
    "non_operating" | "related_party" | "other"

export interface AddBack {
  id: string
  label: string
  amountByYear: Record<string, number> // { "2022":
    120000, ... }
  category: AddBackCategory
  evidence: string // doc/section the agent
    cited
  status: AddBackStatus
}

```

```

export interface PnlYear {
  year: string
  revenue: number
  cogs: number
  grossProfit: number
  opex: Record<string, number>           // category -> amount
  reportedEbitda: number
}

export interface FinancialModel {
  basis: "cash" | "accrual" | "mixed"
  years: PnlYear[]                     // 3+ years, ascending
  ttm?: PnlYear
  addBacks: AddBack[]
  normalizedSdeByYear: Record<string, number> // computed in
                                          code, NOT by LLM
  flags: string[]                      // related-party,
                                          reconciliation gaps, <3yr history, etc.
  extractionConfidence: number         // 0-1
}

export type ValuationStatus = "draft" | "awaiting_seller" |
  "approved"

export interface ValuationOpinion {
  normalizedSde: number                 // the agreed SDE basis
                                          (Year-3 or TTM)
  multipleLow: number
  multipleHigh: number
  appliedMultiple: number
  valuationLow: number
  valuationHigh: number
  recommendedListing: number
  assetFloor?: number
  methodology: string                  // narrative, from Sonnet
  status: ValuationStatus
}

// lib/db/schema/brokerage.ts (shape sketch – finalize columns
// during build)

sellers
  id uuid pk default gen_random_uuid()
  user_id uuid not null unique          // -> auth.users.id
                                          (string FK by value; no cross-schema FK)
  email text not null
  display_name text
  created_at timestampz default now() not null

deals
  id uuid pk default gen_random_uuid()
  owner_user_id uuid not null           // = sellers.user_id
                                          (owner for RLS/app checks)
  assessment_session_id text references
                                          assessment_sessions(session_id) // nullable FK
  business_name text

```

```

industry text // INDUSTRIES.value slug
seller_intent text $type<SellerIntent>()
status text $type<DealStatus>() not null default 'onboarding'
// deal-structure preferences (D6)
asset_vs_stock text $type<AssetVsStock>()
seller_note_tolerance text $type<Tolerance>()
earnout_tolerance text $type<Tolerance>()
real_estate text $type<RealEstate>()
target_close text
preferences_notes text
// computed snapshots
completeness jsonb $type<CompletenessSnapshot>()
valuation jsonb $type<ValuationOpinion>()
created_at / updated_at timestamptz

documents
id uuid pk
deal_id uuid not null references deals(id) on delete cascade
category text $type<DocumentCategory>() not null default
'unclassified'
status text $type<DocumentStatus>() not null default 'uploaded'
storage_path text not null // deals/{dealId}/{
{documentId}/{filename}
original_filename text not null
mime_type text
byte_size integer
sha256 text
classification
jsonb // { category, confidence,
periodLabel, year }
extraction jsonb // raw structured
extraction for this doc
version integer not null default 1
created_at timestamptz

financial_models
id uuid pk
deal_id uuid not null references deals(id) on delete cascade
version integer not null
model jsonb $type<FinancialModel>() not null
recast_status text not null default 'draft' // draft |
approved
created_at timestamptz
unique(deal_id, version)

agreements // engagement letter +
confidentiality protocol (D5)
id uuid pk
deal_id uuid not null references deals(id) on delete cascade
kind text not null // 'engagement' |
'confidentiality'
storage_path text // immutable rendered
snapshot
document_sha256 text

```

```

signed_at timestampz
signed_name text
signer_user_id uuid
signer_ip text
created_at timestampz

approvals                                     // generic approval audit
    (recast, valuation)
id uuid pk
deal_id uuid not null references deals(id) on delete cascade
kind text not null                          // 'recast' |
    'valuation_range'
approved_at timestampz
approver_user_id uuid
snapshot jsonb                               // what exactly was
    approved (model version / range)
created_at timestampz

```

3.2 Migration + RLS (one migration, pnpm db:generate then hand-edit RLS)

- Generate the table DDL from the schema, then append RLS in the **same** migration file, using the idempotent `DO $$ ... EXCEPTION WHEN duplicate_object THEN NULL; END $$`; pattern from `0003_jazzy_dormammu.sql`.
- **RLS posture:** all new tables `ENABLE ROW LEVEL SECURITY`. App reads/writes go through Drizzle (service connection, bypasses RLS) **with mandatory app-level ownership filtering** (where `owner_user_id = user.id` / join through `deal_id`). RLS policies are defense-in-depth for any future direct supabase-js access:
 - authenticated `SELECT` on `deals/documents/financial_models/agreements/approvals` USING `(owner_user_id = auth.uid())` (join deal for child tables).
 - **No anon policies** on any brokerage table.
- **Never** edit an applied migration; this is one new append-only migration. Renumber after `0006`.

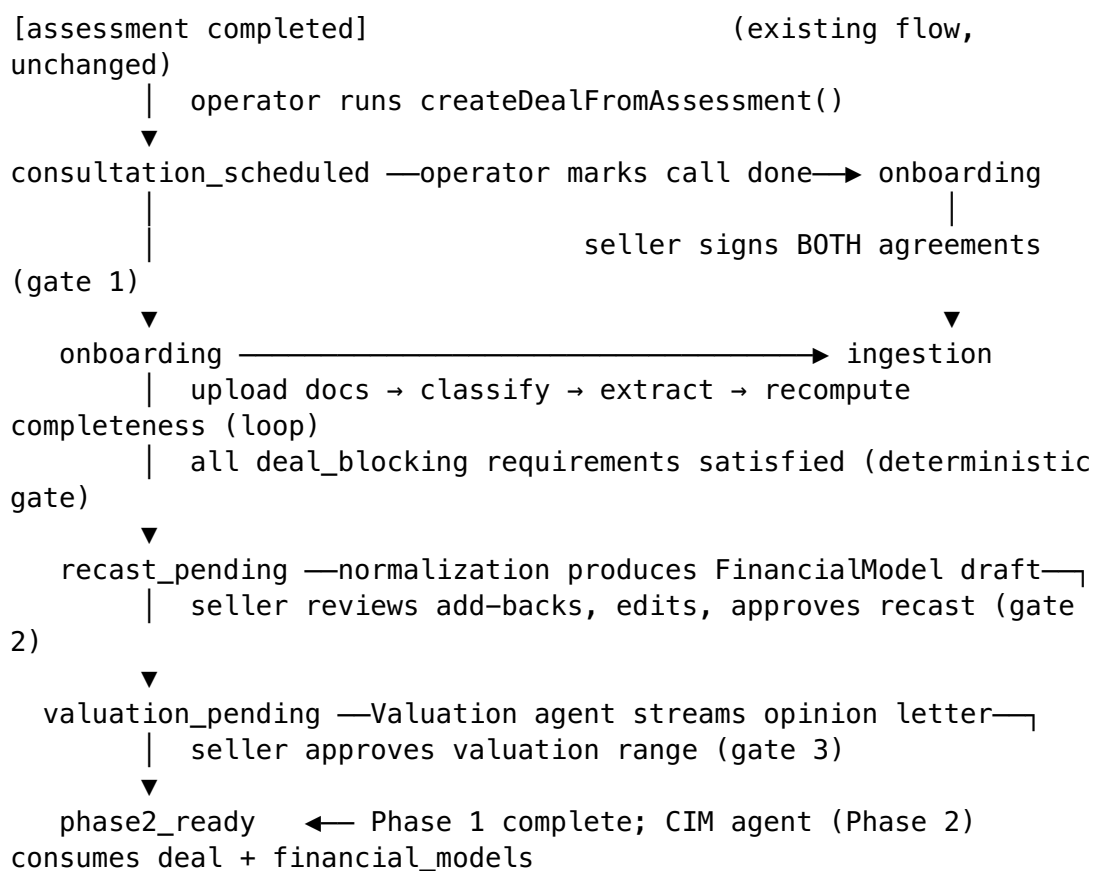
3.3 Supabase Storage

- Create a **private** bucket `deal-documents` (via Supabase dashboard or a one-time setup note — document this in `.env.local.example` / a docs/ note; bucket creation is not a Drizzle migration).
- Storage RLS policy: authenticated may read/write objects whose path prefix maps to a deal they own. Because app flows mint **signed URLs server-side** after a Drizzle ownership check, the policy is belt-and-suspenders; still add it.
- Upload flow uses `createSignedUploadUrl` (server) → client PUT → server records the documents row. Download/preview uses short-lived `createSignedUrl` (server, post ownership check).

3.4 Assessment change for seller_intent (D3)

- Add seller_intent text column to assessment_sessions (same migration or a tiny separate one — your call, but keep RLS intact).
 - Add sellerIntent?: SellerIntent to the relevant TS interface in lib/assessment/session.ts and to the Zod schema in app/api/assessment/session/route.ts (the contract: update **both**).
 - Add a single-select **intent picker** early in the ExitIQ flow (components/exitiq/questions.tsx or a dedicated first step). Persist through the existing saveStage*/POST session path. Keep this change **minimal** — one picker, one column, one Zod field. Do not redesign the assessment.
-

4. The Phase-1 deal state machine



Transitions are written by server routes/actions, never by the client directly. Each transition is idempotent and validates the precondition (e.g. you cannot enter recast_pending unless completeness.dealBlockingSatisfied).

5. Agent pipeline spec

All agents live under `lib/brokerage/agents/` as **server-only** modules. Prompts live in `lib/ai/prompts.ts` (extend it) or a new `lib/brokerage/prompts.ts`. Reuse the single module-level anthropic instance and `SONNET_MODEL/` `HAIKU_MODEL` from `lib/ai/index.ts`. **Never** instantiate a provider per request. Wrap each LLM call in `timed()` (`lib/logger.ts`) and trace phases.

Agent	Trigger	Model / mode	Input	Output
Ingestion (infra)	seller uploads	none	file	Storage row
Classification	<code>document.uploaded</code> (via <code>after()</code>)	Haiku <code>generateObject</code>	filename + first-page/text sniff	document + classification status
Extraction	<code>document.classified</code>	Sonnet <code>generateObject</code>	classified doc content	document status
Completeness	any documents change	deterministic (code) + Haiku for request copy only Sonnet <code>generateObject</code> (structure + add-back candidates) → code recomputes all sums	all deal documents + requirement catalog	deal snapshot
Normalization / Recast	operator/auto when <code>deal_blocking</code> cleared	Sonnet <code>streamText</code> (prose) + code-injected numbers	all extractions	new document (draft)
Valuation	recast approved	Sonnet <code>streamText</code> (prose) + code-injected numbers	approved <code>FinancialModel</code> + <code>INDUSTRIES[].sdeMultiple</code>	deal (approved opinion)

Hard rule on arithmetic: the LLM proposes *categories/candidates/narrative*; **all financial totals (gross profit, EBITDA, normalized SDE, valuation low/high/applied) are computed in pure, unit-tested TypeScript** in `lib/brokerage/recast.ts` and `lib/brokerage/valuation.ts`. Normalized SDE = reported EBITDA + Σ approved add-backs (per year). Valuation = normalized SDE (Year-3 or TTM, per a documented rule) $\times$ `findIndustry(deal.industry).sdeMultiple`. This is the single most important correctness constraint in the build.

Orchestration without a queue: a thin internal route `POST /api/brokerage/deals/[dealId]/advance` (or per-agent routes) runs the next step. Document-triggered steps are kicked from the upload route inside `after()`. Each step reads durable state (`documents.status`, `deals.status`), does its work, writes back, and is safe to call twice. No external job system in Phase 1.

6. API & route contract (new — under `app/api/brokerage/`)

All routes: Zod-validate body/params; resolve the authed user via `lib/supabase/server.ts createClient().auth.getUser()`; **load the deal via Drizzle and assert `owner_user_id === user.id`** (404 if not owned — don't leak existence); structured logs + explicit status codes; PII-safe traces.

```
POST /api/brokerage/documents/upload-url    body {dealId,
filename, mimeType}
                                           → {documentId,
signedUploadUrl, storagePath} (ownership-checked)
POST /api/brokerage/documents              body {dealId,
documentId, sha256, byteSize}
                                           → records row,
status 'uploaded', kicks classify in after()
GET /api/brokerage/deals/[dealId]/documents → list (owner-
scoped) for the intake UI
POST /api/brokerage/deals/[dealId]/classify → (idempotent)
classify any 'uploaded' docs [internal/after]
POST /api/brokerage/deals/[dealId]/extract → (idempotent)
extract any 'classified' docs [internal/after]
GET /api/brokerage/deals/[dealId]/completeness → recompute +
return CompletenessSnapshot
POST /api/brokerage/deals/[dealId]/normalize → run normalization
→ new financial_models draft (gate: deal_blocking)
POST /api/brokerage/deals/[dealId]/recast/approve body
{modelVersion, addBackOverrides?}
                                           → finalize SDE,
write approvals('recast'), status→valuation_pending
POST /api/brokerage/deals/[dealId]/valuation/generate → stream
opinion letter (text/plain), persist in after()
POST /api/brokerage/deals/[dealId]/valuation/approve body
{agreedRange}
                                           → write
approvals('valuation_range'), status→phase2_ready
POST /api/brokerage/agreements/[id]/sign    body {signedName}
                                           → record signed_at/
name/sha256/ip; when both signed, status→ingestion
```

Operator-only (service-role isolated in `lib/supabase/admin.ts`, used ONLY here):

```
(server action, not public)
createSellerAccount({email})           → auth.admin.createUser/
```

```
invite + sellers row
(server action, not public) createDealFromAssessment({sessionId,
sellerEmail, sellerIntent?})
```

```
→ sellers (if needed) + deals row (status consultation_scheduled)
(server action, not public)
markConsultationComplete({dealId}) → status onboarding +
provision agreements rows
```

Streaming routes mirror generate/route.ts: stream text/plain, persist the final text in after(), keep cancellation-safe. The valuation letter **numbers are computed first**, injected into the prompt as fixed facts so the prose can never contradict them.

7. UI surfaces (new real routes under app/(app)/)

Reuse the real auth guard. **Begin unwinding PERSONA:** app/(app)/layout.tsx loads the active deal for user.id (lib/brokerage/queries.ts → getActiveDealForUser) and passes real identity into AppShell (replace the PERSONA.identity.* props with deal/seller values; leave the rest of AppShell untouched for now). If no deal exists for the user, render an empty/“pending consultation” state — never crash.

Route	Purpose	Key elements
app/(app)/onboarding/	agreements sign + deal-structure prefs	Two agreement viewers with typed-name Review & Sign gate; structured prefs form (D6); blocks until both signed
app/(app)/intake/	document upload + completeness	Native file input → signed-URL upload; live document list with per-doc status (uploaded→classifying→classified→extracted), category chips (editable); completeness gap list with severity flags + “Request from seller/CPA” copy
app/(app)/recast/(real)	add-back review + recast approval	3-year normalized P&L table; each add-back row approvable/editable/rejectable; live recomputed SDE; Approve Recast (gate 2). <i>Swap the page wrapper to a new real component; leave the mock RecastStation unused.</i>
app/(app)/valuation/	opinion letter + range approval	Streamed opinion-letter render (reuse report streaming UI patterns); methodology + range + asking-price anchor; Approve Range (gate 3) → success state “Phase 2 ready”

Every surface needs **loading and error states** (contract requirement). Every agent output carries a visible **“Review & Approve”** affordance (agent-native positioning). Match the existing inline-style + CSS-token convention of components/scorta/* (or Tailwind+tokens per AGENTS.md for brand-new files — pick one paradigm per file, don’t mix).

8. Auth & onboarding details

- **Service-role isolation:** create `lib/supabase/admin.ts` (server-only) that builds a `@supabase/supabase-js` client with `SUPABASE_SERVICE_SECRET_KEY`. This is the **only** runtime use of the service key (previously test-only). Justify it in a one-line comment + note it in `CLAUDE.md`. It is imported **only** by the operator onboarding server action. Never import it into a route that a seller can reach, never into a client component.
 - **Account creation** (`createSellerAccount`): `admin-invite` or `createUser` with a temp password + password-reset email (Resend is wired but no-ops without a key — acceptable; operator can set the password in the dashboard for the done-for-you phase). Insert the sellers row mapping `user_id` → `email`.
 - **Agreements** are generated from templates (`lib/brokerage/agreements/*.ts` → `markdown/HTML` with deal-specific variables), rendered to an immutable snapshot, hashed (`sha256`), stored, and signed in-app. Signing records the audit fields and is what flips `onboarding` → `ingestion`.
-

9. Reuse vs. supersede (don't reinvent)

- **Reuse:** `findIndustry()` + `INDUSTRIES[].sdeMultiple` (valuation band), `fmt()` (currency), `isSBAEligible/computeSBASnapshot` (SBA line in the opinion letter), the `streamText+after()` persist pattern, the `Zod-validate-then-safeParse` route pattern, `timed()/logger/traceEvent`.
 - **Supersede (do not feed forward):** the **bracket-based** `getValuationRange` (driven by self-reported buckets) is the GAP/teaser estimate only. The opinion letter must use **precise normalized SDE** from `financial_models`. Keep both; never let the bracket number leak into the formal letter.
-

10. Build sequence (2–3 days)

Build the **spine end-to-end first** so the deal can walk from creation → `phase2_ready` with thin versions of each step, then deepen. Ship green at every checkpoint (`pnpm typecheck && pnpm lint`).

Day 1 — Data spine + onboarding gate

1. `lib/brokerage/types.ts`; `lib/db/schema/brokerage.ts`; export from schema index.
2. One migration: tables + RLS (idempotent D0 blocks) + `assessment_sessions.seller_intent.pnpm db:generate`, hand-edit RLS, `pnpm db:migrate` (session pooler). Add the **seed for deal #1** (Chandan/Palace Kitchen) as a script or migration insert.
3. Storage bucket `deal-documents` + policy (documented).
4. `lib/supabase/admin.ts` + operator actions `createSellerAccount`, `createDealFromAssessment`, `markConsultationComplete` (+ provision agreements rows).

5. `app/(app)/layout.tsx` → load real deal, feed AppShell real identity (PERSONA-unwind, contained).
6. `app/(app)/onboarding/` — agreements sign gate + prefs form; `POST /agreements/[id]/sign`. **Checkpoint:** an operator can create a deal; the seller logs in, signs, lands in ingestion.

Day 2 — Ingestion → completeness → recast

1. Upload: `upload-url` + documents routes + intake UI (real upload to Storage, live list).
2. **Classification** (Haiku `generateObject`) kicked in `after()`; editable category chips.
3. **Extraction** (Sonnet `generateObject`) on classified docs.
4. `lib/brokerage/requirements.ts` catalog + `lib/brokerage/completeness.ts` (deterministic) + `/completeness` route + gap-list UI with severity flags. Deal-blocking gate.
5. `lib/brokerage/recast.ts` (pure SDE math, unit-tested) + **Normalization** (Sonnet `generateObject` proposes add-backs; code computes sums) → `financial_models` draft.
6. `app/(app)/recast/` real UI: add-back approve/edit + `/recast/approve` (gate 2). **Checkpoint:** upload real docs → see classification/extraction → completeness clears → approve a recast → deal in `valuation_pending` with a real `FinancialModel`.

Day 3 — Valuation letter + approval + hardening

1. `lib/brokerage/valuation.ts` (pure: $SDE \times \text{industry band}$ → `range/applied/` `listing/asset floor`).
2. **Valuation agent** `streamText` with code-injected numbers; `/valuation/generate` (`stream` + `persist` in `after()`); `app/(app)/valuation/` streamed render.
3. `/valuation/approve` (gate 3) → `phase2_ready` + success state.
4. Tests (Vitest): `recast.ts`, `valuation.ts`, `completeness.ts`, `requirement catalog`; integration test for the new tables + RLS (mirror `assessment-rls.int.test.ts`). Error/loading states audit. `pnpm typecheck && pnpm lint && pnpm prettier && pnpm test green`. Walk the whole flow.

Cut-lines if behind (drop depth, keep the spine)

- C1: Extraction can return a thinner structure (revenue + EBITDA + owner-comp add-back only); normalization still produces a valid `FinancialModel`.
 - C2: Agreement “documents” can be a single combined engagement+confidentiality doc (one sign event) instead of two.
 - C3: Completeness request copy can be a static template (skip the Haiku draft).
 - C4: Valuation letter can render styled HTML only (print-to-PDF later) — **never** add a PDF lib to hit the deadline.
 - **Never cut:** real schema + RLS, real ownership checks, the deterministic SDE/valuation math + its tests, the three approval gates, green build.
-

11. Testing & Definition of Done (per .claude/CLAUDE.md)

- `pnpm typecheck`, `pnpm lint`, `pnpm prettier clean`.
- **Unit tests** for all pure `lib/brokerage/* math/logic` (recast, valuation, completeness, requirement catalog) — these are the correctness core and must be deterministic.
- **Integration test** for the new tables + RLS (owner can read own deal; cannot read another's; anon blocked), mirroring `lib/db/__tests__/assessment-rls.int.test.ts`.
- Every route handles invalid input / missing rows / non-owner / upstream LLM failure with explicit status + structured log — no crashes.
- Every client surface has loading **and** error states.
- New schema ships RLS in the same migration; `env.mjs` + `.env.local.example` updated together if any var is added (none expected beyond using existing ones).
- No `process.env` outside `env.mjs`; no db import in a client component; no `../..` imports.
- The full flow was actually exercised (create deal → ... → `phase2_ready`) with no console errors.

12. Risks & mitigations

Risk	Mitigation
LLM extraction inaccuracy on messy PDFs	Sonnet for extraction; all math recomputed in code ; everything is operator-editable before any gate; <code>extractionConfidence</code> surfaced; cut-line C1.
Service-role key misuse	Single isolated module <code>lib/supabase/admin.ts</code> , operator-action-only, never seller-reachable, never client.
RLS vs. Drizzle-bypass confusion	Document explicitly: app enforces ownership in code; RLS is defense-in-depth. Owner check in every route.
Scope creep into Phase 2 (CIM)	Hard ceiling at approved valuation range → <code>phase2_ready</code> . CIM is out of scope.
Persona-unwind destabilizing AppShell	Contain the change to identity/header props in the layout; leave the rest of AppShell and all mock stations untouched.
No queue → lost agent steps	Durable state in <code>deals.status/documents.status</code> ; every step idempotent and re-runnable; <code>after()</code> for slow work.
2–3 day overrun	Spine-first sequence + explicit cut-lines C1–C4; never cut schema/RLS/math/tests/green build.

13. Explicit non-goals (this sprint)

- Phase 2 (CIM, teaser, marketplace listings, NDA) and Phase 3 (buyer outreach/qualification).
 - Real DocuSign/e-sign integration (click-sign now; swap later).
 - Real QuickBooks/Plaid/Stripe connectors (the mock ConnectStation is ignored).
 - PDF generation library.
 - Retrofitting or deleting the existing mock stations / PERSONA constant (only the Phase-1 surfaces go real; the rest are addressed in later phases).
 - Public self-serve signup (operator-provisioned only).
-

14. Items needing operator/human input before or during build

1. **Agreement legal text** for the engagement letter + confidentiality protocol templates (provide final copy; the build stubs realistic placeholders until then).
2. **SDE basis rule** for valuation: Year-3 normalized SDE vs. TTM — confirm which is the headline number for the opinion letter (default: **Year-3 normalized SDE**, TTM shown as a cross-check).
3. **Asset-floor + recommended-listing formulas** — confirm the rule (default: listing = midpoint of range; asset floor from equipment/inventory extraction when available, else omitted).
4. **Operator onboarding mechanism** — admin API invite vs. manual Supabase dashboard user creation for the done-for-you phase (default: admin-API server action with manual fallback). ``